



CAD Techniques for NoC-Connected Multi-CGRA Systems

Haoran Wei

ha.wei@mail.utoronto.ca

Dept. of ECE, University of Toronto
Toronto, ON, Canada

Hamas Waqar

hamas.waqar@mail.utoronto.ca

Dept. of ECE, University of Toronto
Toronto, ON, Canada

Omkar Bhilare

omkar.bhilare@mail.utoronto.ca

Dept. of ECE, University of Toronto
Toronto, ON, Canada

Jason H. Anderson

janders@ece.utoronto.ca

Dept. of ECE, University of Toronto
Toronto, ON, Canada

ABSTRACT

Coarse-grained reconfigurable arrays (CGRAs) are programmable hardware platforms that are a means of realizing application accelerators. A CGRA is a 2D array of configurable processing elements (PEs), that connect to one another through programmable interconnects. PEs and interconnects in CGRAs are configurable at the word level, as opposed to at the bit level in FPGAs. A network-on-chip (NoC) is an on-chip communications system comprising routers and links, where data is packet-switched. NoCs provide more scalable communication vs. traditional on-chip busses or crossbars. In this paper, we propose an NoC-connected multi-CGRA system, where multiple modest-sized CGRAs communicate using an NoC. We introduce CAD techniques that partition and place an application across the “distributed” NoC-connected CGRAs. We study the quality and runtime of the CAD techniques on a system having sixteen 4×4 CGRAs connected together using a 2D mesh NoC.

CCS CONCEPTS

• **Hardware** → **Reconfigurable logic and FPGAs; Hardware accelerators; Reconfigurable logic and FPGAs; Interconnect; Electronic design automation.**

KEYWORDS

CGRA, NoC, network-on-chip, CAD, coarse-grained reconfigurable array, placement, routing, architecture

ACM Reference Format:

Haoran Wei, Omkar Bhilare, Hamas Waqar, and Jason H. Anderson. 2024. CAD Techniques for NoC-Connected Multi-CGRA Systems. In *14th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies (HEART'24)* (HEART '24), June 19–21, 2024, Porto, Portugal. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3665283.3665297>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

HEART '24, June 19–21, 2024, Porto, Portugal

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1727-7/24/06

<https://doi.org/10.1145/3665283.3665297>

1 INTRODUCTION

Coarse-grained reconfigurable arrays (CGRAs) are configurable hardware devices that hold promise for implementation of application accelerators. CGRAs comprise a 2D array of processing elements (PEs), each of which typically contains one or more ALUs. The PEs are connected to one another through programmable interconnect, which in many CGRAs is nearest-neighbour style. CGRAs also typically have I/Os or memory units on their periphery. While some CGRAs have been targeted to embedded, low-power scenarios (e.g. [18]), other work has contemplated CGRAs in the high-performance computing (HPC) context (e.g. [5]). Much literature on CGRA architecture and mapping focuses on the former thrust, targeting arrays of tens of PEs. However, for HPC, we envision the possibility of CGRAs having thousands or 10s of thousands of PEs, and in this setting, prior mapping techniques, such as those based on ILP [11], will not be tractable. As with SoCs, multi-processors, and FPGAs, we believe that an alternate on-chip communication approach, namely a network-on-chip (NoC), is a potential means to allow CGRA architecture and CAD to scale into the HPC domain. In this paper, we describe a first step towards such an architecture and CAD flow.

We propose an architecture where individual CGRAs are connected to one another using an NoC. Inter-PE communication within the architecture may either be within one CGRA, or it may be across the NoC. The rationale for exploring such an architecture is multi-fold. First, the mapping problems for the individual CGRAs are independent and can be executed in parallel. Thus, instead of mapping a single large application to a large CGRA, the proposed architecture leads naturally to a divide-and-conquer approach with simpler parallel mapping problems. Second, from an on-chip communication perspective, the proposed architecture may be more scalable than a single large CGRA, making the mapping problem more tractable. Third, there may be area/performance/power benefits of such an architecture, though that investigation is left for future work.

We propose a CAD framework for NoC-connected multi-CGRA systems. This comprises first partitioning a large application into pieces, each to be implemented on an individual CGRA. The partitioning is constrained by the size of the CGRAs, and their I/O capabilities. Following partitioning, we place the partitions within the NoC, so as to maximize NoC throughput as well as latency. Existing CGRA mapping algorithms can be used to map the individual pieces into their respective CGRAs. In this work, for the CGRAs, we use the RIKEN CGRA [4] modelled in the open-source CGRA-ME

framework [12]. The RIKEN CGRA is an elastic CGRA, thereby well-suited to the uncertain latencies associated with use of an NoC. For the NoC, we leverage the open-source work done at Stanford [7], which has been used in prior work that explored integrating an NoC into an FPGA [3]. The CAD requirements here differ from the FPGA scenario, where the individual partitions are pre-defined by the user, rather than being automatically determined in the current work. To the authors' knowledge, there have been few prior attempts to amalgamate CGRAs with an NoC. Our contributions are:

- 1) A proposed CAD flow for NoC-connected multi-CGRA systems. This includes a partitioning and placement flow for breaking a large application into sub-CGRAs and then placing these on the NoC. The partitioning is based on the multi-level paradigm [16]. Two placement approaches are considered: simulated annealing, and integer linear programming (ILP).
- 2) A proposed architecture using a 4×4 mesh NoC, with a 4×4 CGRA located at each NoC router.
- 3) Experimental work assessing NoC throughput, as well as CAD flow run-time and quality.

2 BACKGROUND

2.1 Network-on-Chip (NOC)

An NoC is an on-chip communication fabric that allows modules to communicate with one another through a packet-switched network, similar to traditional computer network. An NoC is comprised of routers, that connect to one another according to a network topology – a mesh in our case. Routers also have ports to connect to local compute modules, which are the senders/receivers of packets across the NoC. One key NoC concept is that of a *flow-control unit (flit)*. A flit is the smallest unit of data sent between NoC routers, with the bitwidth of the flit being the number of parallel wires on the data links between routers. Packets of data are divided into flits, beginning with a head flit, one or more body flits, and finally, a tail flit. A second NoC concept we leverage is that of virtual channels (VCs). In an NoC router, each router port may have more than one VC, and each VC is essentially a buffer inside the port. VCs allow data to be held in such buffers as “staging areas” in high-congestion situations, where a receiver is not able to immediately receive data. The VC concept has been shown to bring considerable throughput benefits [13] by allowing physical NoC links to continue to be used despite congestion. While we elaborate on our specific NoC in the coming sections, the interested reader is referred to [26] for an introduction to NoCs.

2.2 CGRA-ME Framework, the RIKEN CGRA, and CGRA Mapping

We use the open-source CGRA-ME framework [12] to model CGRAs and map applications onto them. CGRA-ME is a modelling and exploration framework for research on CGRA architecture and CAD. It allows one to model a CGRA using a C++ API and map applications onto the modelled CGRA, generating a configuration bitstream. The Verilog for the CGRA can be generated automatically, and, with the bitstream, functionally verified. The Verilog can also

be input to an ASIC synthesis flow, to assess power, performance and area (PPA).

Fig. 1 shows the RIKEN CGRA [4, 6] used in this work, depicted as a 2×2 CGRA in the figure. Each PE contains a floating-point ALU and a switch block (SB). SBs connect to their 8 neighbours and are decoupled from the ALU. The ALU has buffers on its inputs and its output. Surrounding the array are load/store (LS) tiles, for connecting to/from memory. The RIKEN CGRA is an *elastic* CGRA, where the elastic channel is shown at the bottom. 32-bit data is coupled with two control signals: *valid* asserts the upstream sending module has data to send; *stop* can be asserted by the downstream receiving module to exert backpressure. The CGRA operates according to the dataflow paradigm, i.e. operations happen when inputs arrive. Although we use the RIKEN CGRA in this case study, we believe any elastic CGRA would be suitable to connect to an NoC.

Mapping is typically used as a “catch-all” term for CGRA CAD. The input to the mapper is a target CGRA architecture, an application dataflow graph (DFG), as well as user constraints. A DFG is a directed graph where the nodes represent word-wide operations, such as add, multiply, shift, and so on. Edges in the DFG represent the dependencies between such operations. Typically, the DFG would represent the most compute-intensive part of an application, to be mapped onto a CGRA for acceleration. The CGRA mapper places the DFG nodes into PEs, and forms the necessary connections between the PEs to realize the DFG edges. In elastic CGRAs, scheduling of DFG operations is not necessary, owing to the handshaking circuitry.

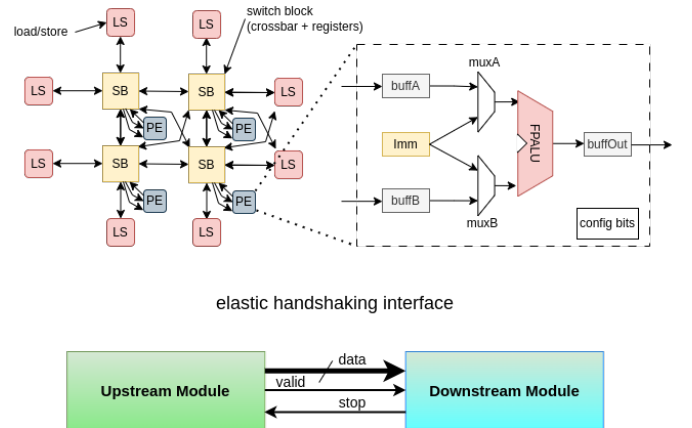


Figure 1: RIKEN CGRA architecture.

3 NETWORK-ON-CHIP (NOC)

Fig. 2a shows the mesh NoC used as the test vehicle in this work. We leverage Becker's open-source NoC routers [7]. All features of the routers are parameterizable, including the number of ports, number of VCs per port, input buffer depth of each VC, network topology, link width, NoC flit format, and more. For all of such parameters, we chose values that we believe to be typical in NoCs. Each of the routers, labelled R0-R15, has 4 compute-node ports, shown as grey circles connected to each router. Each router is associated with one 4×4 RIKEN CGRA – there are 16 of such CGRAs

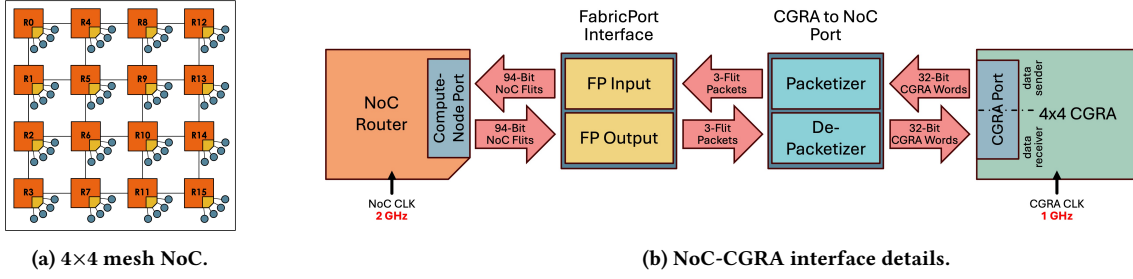


Figure 2: Mesh NoC and NoC-CGRA interface.

in total. Each RIKEN CGRA has 4 NoC ports on its periphery, and also has 4 I/O ports for connections to external memory. The 4 NoC ports are connected to 4 compute-node ports of the router through NoC-CGRA interfaces. In total, across the entire system, there are $16 \times 4 \times 4 = 256$ PEs.

Fig. 2b illustrates how an NoC router port connects to a CGRA port. Each compute-node port of a router is connected to a *fabric port*, and for this, we modified the work by Abdelfattah in [2]. The fabric port is then connected to a CGRA-to-NoC port that we developed. Together, the fabric port and the CGRA-to-NoC port allow the CGRA to send and receive a stream of 32-bit words to/from the NoC network, at an independent clock frequency. We elaborate on the pieces in the subsequent sections.

3.1 NoC Router

Fig. 3 shows the high-level architecture of an NoC router, which is composed of input ports, output ports, an internal crossbar switch with a switch allocator, and a VC allocator. The number of input and output ports depends on the topology of the NoC network and on the number of compute nodes connected to each router. These ports can be classified into link ports and node ports. A router uses link ports to communicate with neighboring routers, and the node ports are used to connect to compute nodes. In our case, the network is a 2D mesh, so each router has 4 link ports except for the peripheral routers. In addition, each router has 4 node ports that are used to communicate with a 4x4 CGRA.

Each VC in the input stage has an input buffer. We experiment with various settings for VC count and buffer depth below. An input flit will be buffered in the pre-assigned VC until its VC allocation and switch allocation are complete. Each input port has a VC controller that reads the look-ahead routing (LAR) information in each buffered head flit and determines the destination output port for it. The controller also performs route computation for each head flit and updates its LAR information for next routing hop. We use dimension-ordered routing (DOR) – a standard and simple NoC routing approach – where flits are first routed towards their destination in the x dimension, then in the y dimension.

A VC allocator assigns each buffered head flit an available VC from its destination output port in a round-robin style and updates the VC identifier in it. Then, all body flits belonging to the same packet will be routed to the same output port and VC in order. There is an output register in each output virtual channel, which buffers an output flit for at least one clock cycle. A switch allocator is responsible for switch resource allocation in a round-robin style

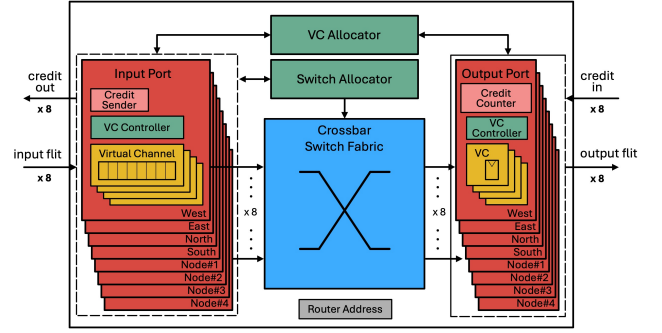


Figure 3: Block diagram of NoC router with 8 ports, 4 VCs per port, and 8-flit buffer depth per VC.

at the packet granularity as well. It manages the crossbar switch to transfer flits from input ports to their destination output ports.

The NoC router uses credit-based flow control – a back-pressure scheme widely used in NoCs. Each input port has a credit sender that sends a credit to the upstream router whenever a flit leaves the input buffer. Each output port has a credit counter that counts down from the depth of VC buffer if the router sends out a flit to a downstream router, and counts up if it receives a credit from the downstream router. An output port will stop sending flits if the credit-count is zero.

3.2 NoC Packet Format

Data transferred in the NoC network is at granularity of flits. Fig. 4 shows the formats of head flits and body flits used by our NoC. This packet format follows an explicit-length format, which explicitly specifies the total number of flits contained in a packet and encodes this information in the head flit. We carefully designed the packet format considering that the CGRA data is 32-bits-wide words, and the desire to have a compact packing of such words into NoC flits and packets. Each flit is 94-bits wide. The NoC link width is equivalent to the flit width. Each flit has a 4 bits of flit control information. The valid bit indicates whether a flit is valid or not. The head bit indicates whether a flit is head flit or body flit. The VC identifier indicates a specific VC to be used for flow control. Our NoC router provides up to 4 VCs at each port, consequently, the VC identifier is 2-bits wide. The link-active bit at the beginning of each flit is used for router power management.

In addition, the head flit contains a 11-bits information used for routing at the granularity of packets. The destination address

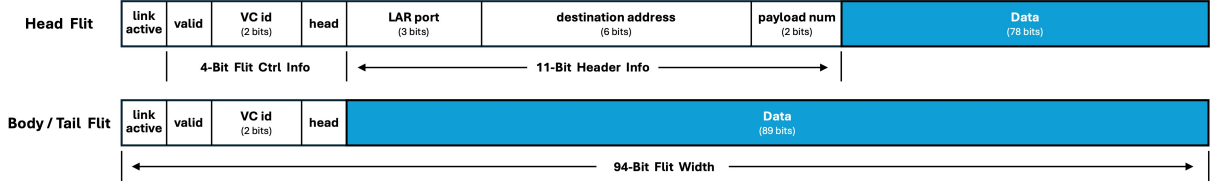


Figure 4: NoC packet format.

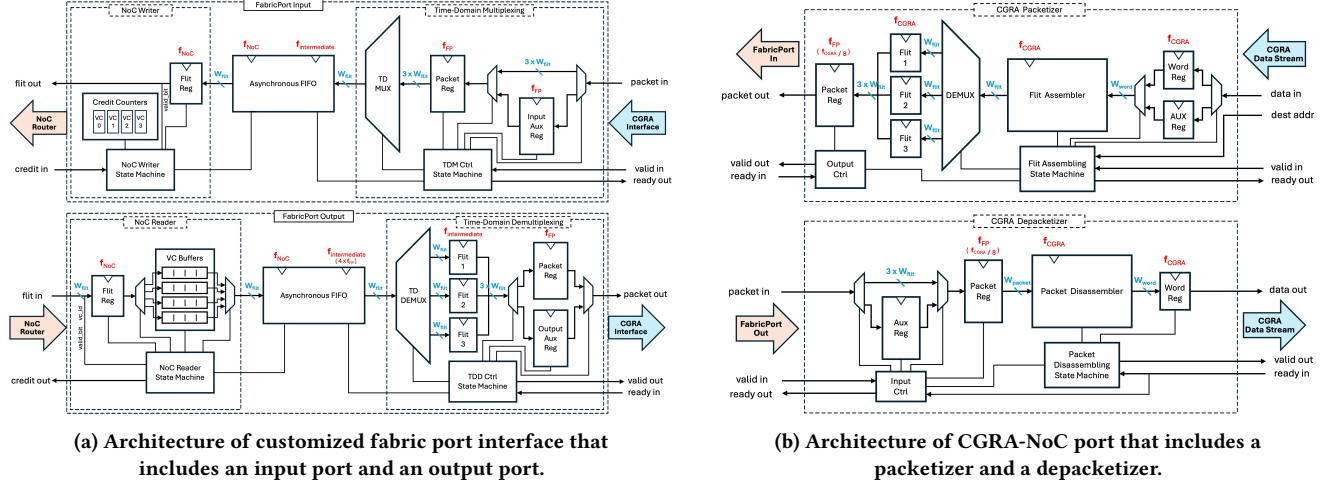


Figure 5: Architecture of customized fabric port interface and CGRA-NoC port.

encodes the destination router and the specific compute port on that router in 6 bits. The look-ahead routing (LAR) port identifier tells the router which output port it should use to send out the packet. This LAR identifier together with the VC identifier are dynamically computed by the previous up-stream router. The payload number explicitly indicates the number of body flits of a packet. In this work, data from the CGRA is packetized as 3-flit packets, which means a packet of CGRA data always has 2 body flits. The width of a 3-flit packet is 282-bits, in which there are 26 control bits in total. A head flit contains 16-bits for control and 78-bits of data, and each body or tail flit contains 5 control bits and 89-bits of data. A packet contains eight 32-bit words of CGRA data. For the case of a 3-flit packet, the control overhead is 9.22% of the data transmitted in the NoC network, which we consider small.

3.3 Customized Fabric Port Module

The fabric port interface is composed of input and output modules. Fig. 5a shows the architecture. The input module takes a packet and disassembles it into 3 flits. Then, it injects them into NoC through a node port of the router. The output module receives 3 flits in a head-body-tail pattern and assembles them into a packet. The fabric port has several key functions:

Data Width Adaptation: Convert wide packets to/from narrower flits (NoC side) via time-domain multiplexing/demultiplexing.

Clock Rate Conversion: Clock-domain-crossing between the CGRA and NoC. Then NoC clock and CGRA clock can have fully independent clock frequencies.

Protocol Translation: Convert between the back-pressure schemes on NoC side and CGRA side. The NoC uses credit-based flow control, whereas the CGRA uses valid/stop back-pressure signals.

Virtual Channel Handling: Read and write NoC virtual channels properly. Since the fabric port is directly connected to a NoC router, it must be able to properly handle the virtual channels in the node port of the router.

Stallability: If the fabric port input sends out a "not ready" signal to its sender, it must be able to accept one more valid data, so that the sender can have time to process the "not ready" signal and stop sending more valid data. An auxiliary input register permits this. Similar functionality is implemented in the output module.

3.4 Custom CGRA-to-NoC Port Module

The CGRA-to-NoC port is composed of a packetizer of NoC flits as an input module, and a depacketizer of CGRA data words as an output module. The architecture is shown in Fig. 5b. The packetizer takes a stream of 32-bit CGRA words from a CGRA port, packs 8 words into a NoC packet, and sends it to the fabric port interface. Conversely, the de-packetizer strips away the control bits of packets received from FabricPort, and splits data into a stream of CGRA words. The CGRA-NoC port module, FabricPort module, and CGRA use the same valid/ready back-pressure scheme, which makes the interfacing straightforward. The CGRA-to-NoC port implements the following 3 functions:

Data Format and Width Conversion: 32-bit words streaming

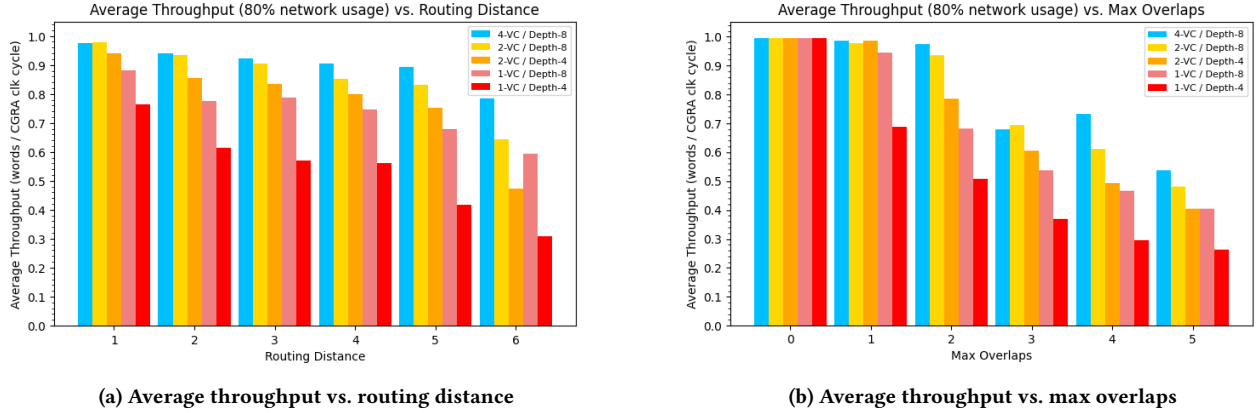


Figure 6: NoC throughput analysis.

from the CGRA are assembled into wide packets containing 8 words, along with destination, routing, and other NoC bits. Similarly packets from the fabric port are disassembled into 32-bit words and streamed into the CGRA.

Clock Rate Conversion: Clock domain crossing between the CGRA and fabric port clocks.

Stallability: Similar to the fabric port, auxiliary registers are used to ensure no data is lost in the event of a stall.

3.5 NoC Throughput Analysis

To understand the NoC throughput dynamics, which influences how we design the CGRA/NoC CAD flow, we simulated the NoC RTL in ModelSim with random sender/receiver pairs. Specifically, we generated 25 pairs of sender and receiver and randomly selected 50 ports within the NoC to connect them, which occupied ~80% of CGRA ports available in the network. We also ensured that the sender and receiver in a pair are not connected to the same router to avoid local routing. We then simulated the NoC for 10,000 CGRA clock cycles, where each sender (mimicking a CGRA) injected a 32-bit word every cycle. The NoC clock was assumed to operate at 2× the CGRA clock. We repeated the simulation 30 times with different random placements of sender/receiver pairs. Furthermore, based on the baseline router design introduced in Section 3.1, we tuned two design parameters – the number of VCs per port and input buffer depth per VC – to understand how they affect the network throughput performance. We opted to test 5 router designs that are different combinations of 4 VCs, 2 VCs and 1 VC with buffer depths of 8 and 4. We repeated the experiment for each of these different router designs, using the same random seeds to reuse the same 30 sets of random sender/receiver placements.

Fig. 6 shows key results of NoC throughput study. Fig. 6a shows how routing distance influences the throughput. NoC throughput averaged across receiver ports over 30 simulations is shown on the y-axis, normalized to 1 (indicating 10,000 words received, or 1 word received per receiver port per CGRA clock cycle). On the x-axis, we see the Manhattan distance between the sender and receiver pairs. Since we use DOR routing algorithm, Manhattan distance is equivalent to routing distance. The key observation is

that NoC throughput decreases as the routing distance goes up. NoC connections between far-apart senders and receivers are more subject to congestion, lowering their throughput. This motivates a goal for our NoC placement flow: to keep senders/receivers as close together as possible. In addition, unsurprisingly, we observed that routers with fewer VCs and smaller VC buffer depths experienced throughput declines, and they are less tolerant to long routing distances.

Fig. 6b shows how routing path overlaps negatively affect the throughput. The normalized average throughput is shown on the y-axis. The x-axis shows the maximum number of overlaps on physical links found on the routing path between each pair of sender and receiver, where zero means there is no overlap at all on a routing path. The key observation is that NoC throughput decreases as overlaps increase, and a sender/receiver pair will obtain a full throughput – 1 word/cycle – if there is no overlap on its routing path. The routing paths that overlap compete for the shared physical links. A high max-overlap value represents a congestion bottleneck on a routing path, where multiple paths vie for a shared link, reducing throughput. Thus, as described below, our CAD flow also aims to minimize such path overlaps. Moreover, we also observe that routers with fewer VCs and smaller VC buffer depths are less tolerant to overlaps. A 4-VC buffer-depth-4 router can guarantee almost full network throughput if path overlap is less than or equal to 2, but for 1-VC depth-8 routers, the throughput significantly drops if more than 1 overlap occurs.

4 CAD SUPPORT FOR NOC-CONNECTED CGRAS

4.1 Overview

As shown in Fig. 7, the proposed CAD framework is a multi-stage approach for mapping an input DFG to the NoC-CGRA hardware. First, the DFG is partitioned into pieces, each of which must fit into a 4×4 CGRA. Next, the partitions are placed on the NoC – that is, each partition is assigned to one of the NoC routers. Finally, the partitions are mapped into the 4×4 CGRAs, using a traditional heuristic or ILP-based mapping approach available in CGRA CAD frameworks such as CGRA-ME.

The modularity of this multi-stage approach, similar to [32], is attractive as a variety of partitioning, placement and CGRA mapping algorithms can be integrated. Another advantage of the proposed CAD flow is its inherent parallelism: Once the large DFG has been partitioned into smaller sub-DFGs and placed into their respective CGRAs, the CGRA mapping processes can be executed in parallel. Specifically, this flow simplifies the mapping process by requiring small sub-DFGs to be mapped onto relatively small CGRAs in parallel. This improves the runtime for the CAD flow relative to mapping a large DFG onto a large CGRA.

4.2 Partitioner

The partitioning problem is quite similar to that of multi-FPGA partitioning. A multi-FPGA partitioner subdivides a large netlist for individual FPGAs, while satisfying constraints like the capacity and IOs for each FPGA. The partitioners utilized in this type of problem can be broadly categorized into simulated-annealing-based [10, 21] and multi-level min-cut-based partitioners [8, 17].

The multi-level partitioners are known to provide high-quality results compared to their counterpart methods [30]. We have utilized a customized version of TritonPart [8] – an open-source K-way multi-level partitioner, primarily due to its capability to partition based on multiple constraints. Our partitioning problem involves various constraints, such as:

- [A] **Capacity constraint** [≤ 16 ALU operations]: A compute node (4×4 CGRA) has 16 PEs, allowing a maximum of 16 ALU operations per CGRA. So, there should be no more than 16 ALU operations in a partitioned sub-DFG.
- [B] **I/O constraints** [≤ 4 memory read/write operations]: A node only has 4 I/O ports, so a partitioned sub-DFG can have up to 4 memory operations.
- [C] **NoC ports constraint** [≤ 4 connections to other partitions]: Since each NoC router has only 4 node ports connected to a node, a partitioned sub-DFG should not have more than 4 crossing edges to other partitions.

The capacity [A] and I/O [B] constraints are incorporated into our modified version of TritonPart. We integrated these constraints by modifying TritonPart's *UpperVertexBalance()* function, which guides the partitioner in deciding the maximum number of different operations in a sub-partition. Introducing different vertex types with differing balance requirements converts a single-constraint partitioning problem into a multi-constraint one. We achieved this by utilizing TritonPart's multi-dimensional vertex weight feature. We also have secondary constraints, such as that constant inputs to ALU operations must reside in the same partition. To accomplish this, we used TritonPart's group constraint feature, which ensures that vertices of the same group belong to the same partition in a graph.

The partitioning algorithm is shown in Algorithm 1. Each partition can have up to 4 memory and 16 ALU operations. Thus, the initial required number of partitions is computed based on these parameters (line 2). The *Num_of_parts* and *balance* parameters indirectly influence the NoC ports constraint or crossing edges [C]. The balance factor determines the allowable imbalance between partitions. TritonPart runs for *max_iterations* (line 4). If the partitioner fails to satisfy any constraint, we rerun it with incremented

Num_of_parts and *balance*. If an NoC port constraint is satisfied but a memory operation constraint is not due to a lower balance value, we apply a quick fix using the *Mem_Distribution_Opt()* function. This function moves excessive memory operations from one partition to another in an attempt to legalize the result.

4.3 NoC Placement by Simulated Annealing

This stage places the (up to 16) partitions within the 4×4 mesh NoC. As a first-cut placement approach, we use simulated annealing (SA). The annealing cost function has two components: 1) Minimize the half-perimeter bounding box wirelength (HPWL) between connected partitions. 2) Minimize overlap on physical NoC links. The former is standard in any SA-based placer. The latter is unique to placement within our NoC. Specifically, as the NoC uses dimension-ordered routing (DOR), it is possible, from the placement, to identify physical links that are used by more than one sender/receiver pair. Consider the example shown in Fig. 8 where R15 sends to R5, and R14 sends to R4. Because, of DOR, we can detect the overlap on the physical link between R14 and R13, shown in bold. Note that physical NoC links have directionality, e.g. there are separate links in the East $\rightarrow$ West vs. West $\rightarrow$ East directions. As such, the connection from R1 to R9 in the example does not contribute to NoC link overlap. The current implementation of SA prioritizes the HPWL cost over the overlap cost. We suppose that minimizing the overall HPWL in the placement will inherently reduce the likelihood of overlap occurring. Therefore, the HPWL cost function in the SA implementation is assigned a weight of 0.7, while the overlap cost function is assigned a weight of 0.3. The current implementation of the SA placer initializes the temperature to 10. During subsequent passes, the temperature decreases by 10%. This approach compels the algorithm, in the initial passes, to accept some undesirable swaps. As temperature decreases, the acceptance of undesirable moves becomes less likely, facilitating fine-tuning of the placement for the best achievable outcome that minimizes both HPWL and overlaps. The temperature of 10 was chosen initially because it provided high initial acceptability for undesirable swaps, considering the placement's cost.

4.4 NoC Placement by Integer Linear Programming (ILP)

As a means to assess the "goodness" of our SA-based placement, we also implemented an ILP approach that is able to find the optimal HPWL, as well as minimizing NoC link overlaps. For the first part of the formulation that minimizes HPWL, our approach is adapted from Kim and Kim [19]. We begin by introducing some ILP variables:

- $\forall i \in \text{Partitions}$: x_i, y_i are integer variables representing the x and y position of partition i on the NoC mesh. x_i and y_i are restricted to values ≥ 0 and ≤ 3 in our NoC.
- $\forall i, j \in \text{Partitions}, s.t. i \neq j$: $rx_{i,j}, rx_{j,i}, ry_{i,j}, ry_{j,i}$ are binary relaxation variables.
- $\forall n \in \text{Connections}$: lx_n, hx_n, ly_n, hy_n are integer variables representing the lowest x , highest x , lowest y , and highest y , respectively, of any of the two partitions part of the connection n .

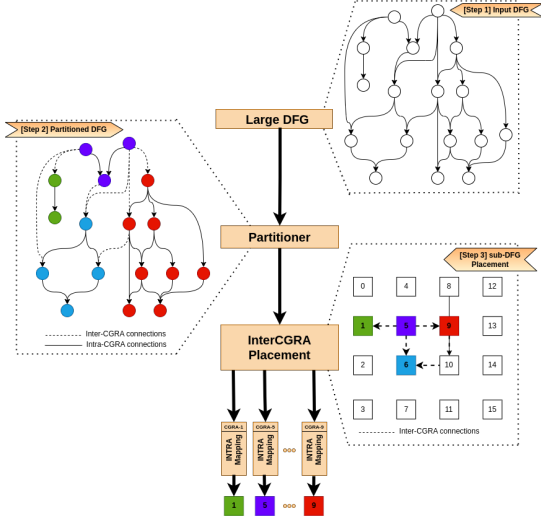


Figure 7: CAD flow for the NoC-CGRA architecture.

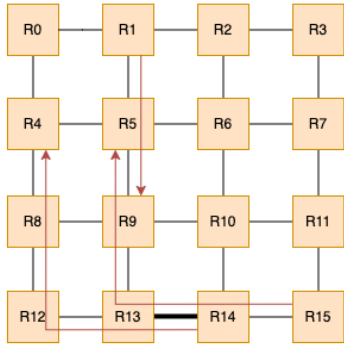


Figure 8: Illustration of NoC link overlap.

- tx_n, rx_n : integers representing the partitions sending and receiving NoC connection n , respectively.

The primary ILP objective is to minimize the sum of the HPWL of all connections:

$$\phi = \sum_{n \in \text{Connections}} (hx_n - lx_n) + (hy_n - ly_n) \quad (1)$$

Subject to constraints:

$$\forall_{i,j \in \text{Partitions}, s.t. i \neq j : x_i < x_j + rx_{i,j} \cdot M \quad (2)$$

$$\forall_{i,j \in \text{Partitions}, s.t. i \neq j : x_j < x_i + rx_{j,i} \cdot M \quad (3)$$

$$\forall_{i,j \in \text{Partitions}, s.t. i \neq j : y_i < y_j + ry_{i,j} \cdot M \quad (4)$$

$$\forall_{i,j \in \text{Partitions}, s.t. i \neq j : y_j < y_i + ry_{j,i} \cdot M \quad (5)$$

where M is a large scalar constant, such as 100. The first constraint checks whether partition i is placed left of j in the NoC. If not, the relation variable $rx_{i,j}$ must be set to 1. The second constraint checks if i is placed right of j , and so on, for the y dimension

Algorithm 1 NoC-CGRA partitioning approach.

```

1: procedure NoC_CGRA_PARTITIONER (INPUT DFG, BALANCE, SEED)
2:    $Num\_of\_parts \leftarrow \text{Max}(\frac{\#MEMOp}{4}, \frac{\#ALUOp}{16})$ ;
3:   while  $max\_iterations \neq 0$  do
4:      $Success \leftarrow \text{run\_TritonPart}(Num\_of\_parts,$ 
5:        $balance, group\_constraints, seed);$ 
6:      $Mem\_Distribution\_Opt();$ 
7:     if  $Success$  then
8:        $exit();$ 
9:     else
10:       $Increase\_Balance(balance);$ 
11:       $Num\_of\_parts \leftarrow Num\_of\_parts + 1$ ;
12:    end if
13:     $max\_iteration \leftarrow max\_iteration - 1$ ;
14:  end while
15: end procedure

```

as well.

$$\forall_{i,j \in \text{Partitions}, s.t. i \neq j : rx_{i,j} + rx_{j,i} + ry_{i,j} + ry_{j,i} \leq 3 \quad (6)$$

which implies at most three relaxation variables are 1, ensuring that two partitions are not placed on top of one another – all four relaxation variables for a pair of partitions cannot be simultaneously 1.

The next constraints set the connection bounding-box variables:

$$\forall_{n \in \text{Connections}} lx_n \geq x_{tx_n}, lx_n \geq x_{rx_n}, hx_n \leq x_{tx_n}, hx_n \leq x_{rx_n} \quad (7)$$

$$\forall_{n \in \text{Connections}} ly_n \geq y_{tx_n}, ly_n \geq y_{rx_n}, hy_n \leq y_{tx_n}, hy_n \leq y_{rx_n} \quad (8)$$

The above formulation will find a placement on the NoC having the optimal HPWL. As our NoC uses predictable DOR, first in the x dimension and then in the y dimension, we were able to extend the ILP formulation to minimize NoC overlaps among connections by introducing new variables:

- $\forall_{n,m \in \text{Connections}, s.t. n \neq m} YOverlap_{n,m}, XOverlap_{n,m}$: binary variables that indicate whether the paths to the receivers of connections of n and m overlap in the y dimension, and whether the paths from the senders of n and m overlap in the x dimension, respectively.
- $\forall_{n,m \in \text{Connections}, s.t. n \neq m} YInterval_{n,m}, XInterval_{n,m}$: integer variables representing the length of the overlapping interval of connections n and m in the y dimension and x dimension, respectively. These intervals are counts of the number of overlapped physical NoC links.

The new ILP objective function is to minimize HPWL and overlaps, with the latter having lower weight:

$$\theta = \phi + 0.01 \cdot \sum_{n,m \in \text{Connections}, s.t. n \neq m} YOverlap_{n,m} \cdot YInterval_{n,m} + XOverlap_{n,m} \cdot XInterval_{n,m} \quad (9)$$

We introduce constraints to set the $YOverlap$ and $XOverlap$ binary variables as follows, using $YOverlap$ for illustration: For two

connections n, m , $YOverlap_{n,m}$ is set to 1 if and only if: 1) $rx_n == rx_m$, i.e. the loads of the two connections are in the same column of the NoC; and 2) The direction of the connections is aligned in the y dimension, i.e. both connections are from S→N or both are from N→S. An analogous approach is used for the $XOverlap$ variables, checking whether the drivers on the connections are in the same row, and whether the connections are directionally East/West aligned.

4.5 Intra-CGRA Mapping

The final stage of the framework is to map each partition into its 4×4 CGRA. For this, we have utilized the clustered mapper [27] included in the CGRA-ME framework. The clustered mapper uses SA for intra-CGRA placement, and uses PathFinder [23] for intra-CGRA routing. A crossing edge between two partitions is connected using NoC-ports and NoC physical links in our architecture, but there is no notion of such NoC ports in CGRA-ME. Consequently, we have designated certain RIKEN memory ports as NoC ports to obtain valid mapping results – the handshaking in the elastic RIKEN CGRA is compatible with the handshaking interface of the NoC-to-CGRA port described earlier. As will be demonstrated in the experimental study, the majority of the CAD run-time benefit is observed in this stage, as mapping small-sized sub-DFGs in parallel is more efficient than mapping a large DFG onto a bigger CGRA.

5 EXPERIMENTAL STUDY

5.1 Methodology

We evaluated the NoC-CGRA CAD framework targeting the proposed architecture using 19 HPC benchmarks [14], including balanced and unbalanced DFGs of convolution, stencil computation, and FFT-Radix 4. Various unrolling factors, from 1 to 10, were applied to each benchmark. For all the benchmarks, the CAD execution time of the partitioner, placer and intra-CGRA mapping stages was averaged across five runs using different random seeds on an Intel Xeon server. ILP placement was computed with the Gurobi commercial solver ver. 9.5.2, with a timeout of 1800 seconds. For the final intra-CGRA mapping, only the partition that took the longest time contributes to the total NoC-CGRA CAD runtime. This is because the mappings of the individual partitions are entirely independent and can execute in parallel. We similarly collected the average CAD runtime of the clustered mapper for the same benchmarks targeting a large 16×16 RIKEN CGRA with the same amount of I/Os as the NoC-CGRA architecture. The 16×16 RIKEN CGRA was also modelled in CGRA-ME.

5.2 Characterization of the CAD Algorithms and Runtime

Partitioner: Columns 2-4 in Table 1 show the partitioning results. Our benchmarks have on average ~ 80 DFG nodes going up to ~ 200 nodes, and the multi-level partitioner was able to find a valid solution with total overall average NoC connection cost of ~ 6 . A NoC connection is defined as a crossing edge between two compute nodes (two CGRAs on the NoC). The partitioning time accounts for an average ~ 99 percent of NoC-CGRA total CAD runtime. We observed that the number of initial partitions and balance

constraint had a significant impact on runtime. Particularly for our moderate-size benchmarks, in the case of a small value of K for a K -way partition with a lower balance, the partitioner converges very quickly due to the smaller solution space. However, when K is large (i.e., more partitions) and there is a higher imbalance, the partitioner has a long execution time. One of the limitations of TritonPart is the lack of direct control over cut size for a single partition. As a result, controlling the I/O constraints was the most difficult challenge. We have observed that the ILP formulation used in initial partitioning within TritonPart for a K -way multi-level partition significantly slows down the framework. As a result, we disabled this feature of TritonPart and obtained $\sim 90\times$ average runtime improvement with a slight impact on the cost metric. We had to enable ILP in the initial partitioning stage of TritonPart for two benchmarks to obtain a valid solution (bottom two benchmarks in table), and as a result, the runtime was significantly longer in those cases. However, this can be addressed in future by modifying the multi-level partitioning algorithm to consider individual partition cut sizes.

Placer: Columns 5 and 7 show the HWPL results for the SA and ILP-based placers, respectively; columns 6 and 8 give the total NoC link overlap results. In most of our benchmarks, both SA-based and ILP placers found placement solutions with nearly zero overlap and an average total HPWL of approximately 7. We observed that the SA-based placer achieved almost identical results to the ILP placer due to the moderate size of our benchmarks. However, for larger or more complex benchmarks such as FFT_Radix4_nounroll benchmark, we noticed that the SA placer could not find the ideal placement as determined by the ILP placer. It is important to note a significant caveat: the ILP partitioner times out for larger benchmarks, meaning that the optimal solution cannot always be determined within a reasonable time frame. The timeout for large DFGs is due to the inherent dependencies of ILP on branch and bound optimizations. While these optimizations aim to reduce ILP's runtime, the sheer number of nodes in the DFGs necessitates more time for ILP to achieve an optimal placement. For example, our experiments showed that ILP required 35400 seconds (~ 10 hours) to find an optimal placement for the conv_unroll6_Balance benchmark, far exceeding our timeout limit. Therefore, ILP is not suitable for placement of large benchmarks in the NoC architecture. Since these benchmarks are moderately sized, it appears that most of the CGRAs are underutilized (partition counts are < 16), and perhaps this is the reason the SA placement achieved optimal HPWL for most benchmarks.

Clustered Mapper: Columns 9, 10, and 11 of the table given the runtime of the clustered mapper targeting a single benchmark to a large 16×16 CGRA, the total NoC-CGRA CAD flow runtime (using the SA placer), and the speedup ratio, respectively. Although not shown in the table, the average runtime ratio of the clustered mapper for a large CGRA to sub-DFG mapping onto a 4×4 CGRA is $\sim 430:1$. The mean total runtime speedup of the our CAD framework vs. typical mapping on large CGRA was $\sim 33\times$. We observed that in each type of benchmark, the highest speedup occurs in the no-unrolled case. As unrolling increases in a benchmark, the complexity for the partitioner starts increasing, and due to the partitioning runtime, speedup decays. However, even in most unrolled

Benchmarks	Partitioner			Simulated Annealing (SA) Placer		ILP Placer		Cluster Mapping		
	# DFG nodes	# Min Partition	# NoC Connections	Total HPWL Wire-length	Total Link Overlap	Total HPWL Wire-length	Total Link Overlap	16×16 RIKEN CGRA Avg CAD run-time (Seconds)	NoC-CGRA run-time (Seconds)	Speedup
conv_nounroll_Balance	36	3	3	4	0	4	0	60.25	0.64	93.50
conv_unroll2_Balance	72	6	6	7	0	7	0	59.21	1.08	54.94
conv_unroll3_Balance	108	8	10	12	1	12	1	71.53	1.65	43.40
conv_unroll6_Balance	216	15	16	19	2	19	TO	91.69	59.42	1.54
conv_nounroll_nonBalance	36	3	3	4	0	4	0	59.72	0.59	100.79
conv_nounroll2_nonBalance	72	5	5	6	0	6	0	64.51	1.05	61.27
conv_nounroll3_nonBalance	108	8	10	12	1	12	0	66.62	1.60	41.65
stencil_nounroll_Balance	20	2	1	1	0	1	0	61.43	0.51	121.29
stencil_unroll2_Balance	40	3	2	2	0	2	0	61.68	0.61	101.60
stencil_unroll3_Balance	60	5	5	5	1	5	1	63.31	0.83	76.16
stencil_unroll4_Balance	80	6	5	6	0	6	0	70.53	1.26	55.99
stencil_unroll5_Balance	100	8	6	6	0	6	0	60.53	1.75	34.63
stencil_nounroll_nonBalance	20	2	1	1	0	1	0	59.33	0.46	130.24
stencil_unroll2_nonBalance	40	3	2	2	0	2	0	59.73	0.68	87.51
stencil_unroll3_nonBalance	60	5	5	5	0	5	0	60.75	0.92	66.30
stencil_unroll4_nonBalance	80	6	6	6	0	6	0	61.35	1.28	47.86
stencil_unroll5_nonBalance	100	8	8	8	1	8	1	77.32	1.65	46.94
stencil_unroll10_nonBalance**	200	15	12	12	0	12	TO	89.61	96.96	0.92
FFT_Radix4_nounroll**	62	7	14	22	10	22	8	78.15	346.57	0.23
Average	79.47	6.21	6.32	7.37	0.84	7.37	0.65	66.58	2.04	32.62

Table 1: NoC CGRA CAD results. [**] >These benchmarks were run with ILP enabled in the partitioner’s initial placement, along with the balance and num_of_parts parameters obtained from a hit case of a larger regression. [TO] >Timed Out, 1800 seconds limit for ILP-placer.

cases such as *Stencil_unroll5_Balance*, the speedup was found to be around $\sim 55\times$.

The main takeaway is that there are significant runtime benefits arising from the ability to break a large application into NoC-connected partitions, each of which can be mapped in parallel.

5.3 Area Characterization

To gain some perspective on the area cost of the NoC relative to the CGRA, we used Synopsys Design Compiler to synthesize both structures into 45nm standard cells, targeting the NanGate library [1] with minimum-area optimization. One 4×4 RIKEN CGRA consumed $\sim 377K\mu m^2$ solely for the core tiles and interconnect (i.e. not including the LS tiles). The NoC router, fabric ports, and CGRA-to-NoC port modules that are coupled with each of 4×4 CGRAs consumed $\sim 200.2K\mu m^2$, for the 1-VC router with depth-4 buffers scenario, and 8 ports – 4 ports for the mesh connectivity, and 4 ports to the CGRA. Of this total area, 25% was consumed by the router, 55% by the fabric ports, and 20% by the CGRA-to-NoC ports. While this provides a rough area evaluation, the NoC components used in this study are quite generic, and as such, we believe significant area optimizations are possible if they are tailored to a specific implementation.

6 RELATED WORK

The authors in [28] [22] use NoC routers to interconnect small CGRA clusters. The CGRA clusters in their work are ringlets consisting of only 4 PEs, compared to our 4×4 CGRA tiles. This gives us more flexibility in mapping a portion of an input DFG to a CGRA, and owing to our NoC-CGRA interface block, we can packetize and

depacketize more flits (and thereby data words) into a single data packet rather than using a single-packet approach. Furthermore, our CAD framework takes advantage of parallel mapping of DFG partitions to smaller CGRAs.

Some implementations of the traditional CGRA CAD framework employ partitioning to reduce CAD run-time and as well as to improve quality of result. In [32], the authors used spectral clustering [31] to divide a large DFG into small DFGs, minimizing the cut size. Our approach, on the other hand, incorporates multi-objective partitioning. Other implementations of CGRA CAD use clustering as a partitioning method for different objectives, such as optimizing placement [15], routing [25], and scheduling [20]. Our framework aims to leverage parallelism in mapping individual partitions onto separate NoC-connected CGRAs.

Various CAD implementations have utilized multi-objective partitioning in multi-FPGA systems [9, 24, 29, 33], which connect multiple FPGAs via I/Os. The constraints in such implementations are similar to ours, with the main difference being the availability of time domain multiplexing (TDM) for I/Os in FPGAs. This allows for the combining of several crossing signals before sending them to different FPGAs, simplifying the partitioning algorithm. TDM of an NoC node port within an CGRA and its implications on area/power are left to future research.

7 CONCLUSIONS AND FUTURE WORK

We proposed a CAD flow for NoC-connected multi-CGRA systems, comprising partitioning a large application into pieces, placing the pieces on an NoC, and then mapping the pieces into individual CGRAs. Partitioning was implemented by adapting an

open-source multi-level partitioning to respect the constraints of the individual CGRAs. NoC placement was implemented in two ways: using simulated annealing (SA) and also using ILP. Other placers minimize both NoC wirelength, as well as congestion on physical NoC links between routers, given the NoC's DOR algorithm. Results showed the SA placer achieved the optimal or close-to optimal HPWL for most benchmarks. Run-times for the proposed CAD flow were $\sim 33\times$ faster than mapping a single application onto a single large CGRA.

Future directions are many, involving implementation of a custom NoC router that is small and simple, comparisons between different router implementations, optimizations of NoC-CGRA interface components, experimenting with different routing algorithms and topologies, as well as a comprehensive power/performance/area (PPA) analysis of NoC-connected multi-CGRA systems vs. large monolithic CGRAs.

REFERENCES

- [1] 2023. The NanGate FreePDK45 Open Cell Library.
- [2] Mohamed Abdelfattah. 2016. Embedded Networks on Chip for Field-Programmable Gate Arrays. *Ph.D. Thesis, University of Toronto* (2016).
- [3] Mohamed S. Abdelfattah and Vaughn Betz. 2013. The power of communication: Energy-efficient NOCS for FPGAs. In *FPL*.
- [4] Boma Adhi, Carlos Cortes, Yiyu Tan, Takuya Kojima, Artur Podobas, and Kentaro Sano. 2022. The Cost of Flexibility: Embedded versus Discrete Routers in CGRAs for HPC. In *IEEE CLUSTER*.
- [5] Boma A. Adhi, Carlos Cortes, Tomohiro Ueno, Yiyu Tan, Takuya Kojima, Artur Podobas, and Kentaro Sano. 2022. Exploring Inter-tile Connectivity for HPC-oriented CGRA with Lower Resource Usage. In *IEEE FPT*.
- [6] Jason Anderson, Boma Adhi, Carlos Cortes, Emanuele Del Sozzo, Omar Ragheb, and Kentaro Sano. 2023. Exploration of Compute vs. Interconnect Tradeoffs in CGRAs for HPC. In *HEART*. 59–68.
- [7] Daniel Becker. 2012. Efficient Microarchitecture for Network-on-Chip Routers. *Ph.D. Thesis, Stanford University* (2012).
- [8] Ismail Bustany, Grigor Gasparyan, Andrew B. Kahng, Ioannis Koutis, Bodhisatta Pramanik, and Zhiang Wang. 2023. An Open-Source Constraints-Driven General Partitioning Multi-Tool for VLSI Physical Design. In *IEEE/ACM ICCAD*.
- [9] Ming-Hung Chen, Yao-Wen Chang, and Jun-Jie Wang. 2021. Performance-Driven Simultaneous Partitioning and Routing for Multi-FPGA Systems. In *ACM/IEEE DAC*. 1129–1134.
- [10] Tao Chen. 2004. *Multi-FPGA Partitioning Using Simulated Annealing*. Master's thesis. University of Cincinnati. http://rave.ohiolink.edu/etdc/view?acc_num=ucin1077766831
- [11] S. Alexander Chin and Jason Helge Anderson. 2018. An architecture-agnostic integer linear programming approach to CGRA mapping. In *IEEE/ACM DAC*.
- [12] S. Alexander Chin, Noriaki Sakamoto, Allan Rui, Jim Zhao, Jin Hee Kim, Yuko Hara-Azumi, and Jason Anderson. 2017. CGRA-ME: A unified framework for CGRA modelling and exploration. In *IEEE ASAP*. 184–189.
- [13] William J. Dally and Hiromichi Aoki. 1993. Deadlock-Free Adaptive Routing in Multicomputer Networks Using Virtual Channels. *IEEE Trans. Parallel Distributed Syst.* 4, 4 (1993), 466–475.
- [14] Jens Domke, Kazuaki Matsumura, Mohamed Wahib, Haoyu Zhang, Keita Yashima, Toshiaki Tsuchikawa, Yohei Tsuji, Artur Podobas, and Satoshi Matsuoka. 2019. Double-Precision FPGAs in High-Performance Computing: An Embarrassment of Riches?. In *IEEE IPDPS*. 78–88.
- [15] Stephen Friedman, Allan Carroll, Brian Van Essen, Benjamin Ylvisaker, Carl Ebeling, and Scott Hauck. 2009. SPR: an architecture-adaptive CGRA mapping tool. In *ACM FPGA*. 191–200.
- [16] G. Karypis, R. Aggarwal, V. Kumar, and S. Shekhar. 1999. Multilevel hypergraph partitioning: applications in VLSI domain. *IEEE TTVLSI* 7, 1 (1999), 69–79.
- [17] George Karypis and Vipin Kumar. 1999. Multilevel k-way hypergraph partitioning. In *ACM/IEEE DAC*. 343–348.
- [18] Changmoo Kim, Moo-Kyoung Chung, Yeon-Gon Cho, Mario Konijnenburg, Soojung Ryu, and Jeongwook Kim. 2014. ULP-SRP: Ultra Low-Power Samsung Reconfigurable Processor for Biomedical Applications. *ACM Trans. Reconfigurable Technol. Syst.* 7, 3 (2014), 22:1–22:15.
- [19] Jae-Gon Kim and Yeong-Dae Kim. 2003. A linear programming-based algorithm for floorplanning in VLSI design. *IEEE TCAD* 22, 5 (2003), 584–592.
- [20] Leibo Liu, Xingchen Man, Jianfeng Zhu, Shouyi Yin, and Shaojun Wei. 2020. Pattern-Based Dynamic Compilation System for CGRAs With Online Configuration Transformation. *IEEE Trans. on Parallel and Distributed Systems* 31, 12 (2020), 2981–2994.
- [21] Mohammadmahdi Mazraei. 2023. *Multi-FPGA Application Partitioning and Infrastructure*. Master's thesis. University of Toronto (Canada). <https://tspace.library.utoronto.ca/handle/1807/128122>
- [22] Somnath Mazumdar, Alberto Scionti, Antoni Portero, Jan Martinović, and Olivier Terzo. 2018. A Scalable and Low-Power FPGA-Aware Network-on-Chip Architecture. In *Complex, Intelligent, and Software Intensive Systems*, Leonard Barolli and Olivier Terzo (Eds.). Springer International Publishing, Cham, 407–420.
- [23] Larry McMurchie and Carl Ebeling. 1995. Pathfinder: A negotiation-based performance-driven router for FPGAs. In *ACM FPGA*. 111–117.
- [24] Iyad Ouass, Sriram Govindarajan, Vinoo Srinivasan, Meenakshi Kaul, and Ranga Vemuri. 1998. An integrated partitioning and synthesis system for dynamically reconfigurable Multi-FPGA architectures. In *Parallel and Distributed Processing*, José Rolim (Ed.). Springer Berlin Heidelberg, Berlin, Heidelberg, 31–36.
- [25] Hyunchul Park, Kevin Fan, Scott Mahlke, Taewook Oh, Heeseok Kim, and Hongseok Kim. 2008. Edge-centric modulo scheduling for coarse-grained reconfigurable architectures. In *2008 International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 166–176.
- [26] Li-Shiuan Peh and Natalie Enright Jerger. 2009. *On-Chip Networks*. Morgan and Claypool Publishers.
- [27] Omar Ragheb and Jason Helge Anderson. 2024. CLUMAP: Clustered Mapper for CGRAs with Predication. In *IEEE/ACM DAC*.
- [28] Alberto Scionti, Somnath Mazumdar, and Antoni Portero. 2018. Towards a Scalable Software Defined Network-on-Chip for Next Generation Cloud. *Sensors* 18, 7 (2018).
- [29] Navaratnasothie Selvakumaran, Abhishek Ranjan, Salil Raje, and George Karypis. 2004. Multi-resource aware partitioning algorithms for FPGAs with heterogeneous resources. In *IEEE/ACM DAC*. 741–746.
- [30] S. Subramanian, D.M. Rao, and P.A. Wilsey. 2000. Study of a multilevel approach to partitioning for parallel logic simulation. In *IEEE IPDPS*. 833–838.
- [31] Ulrike von Luxburg. 2007. A Tutorial on Spectral Clustering. *Statistics and Computing* 17(4), 2007. (2007). [arXiv:arXiv:0711.0189](https://arxiv.org/abs/0711.0189)
- [32] Dhananjaya Wijerathne, Zhaoying Li, Thilini Kaushalya Bandara, and Tulika Mitra. 2022. PANORAMA: divide-and-conquer approach for mapping complex loop kernels on CGRA. In *IEEE/ACM DAC*. 127–132.
- [33] Dan Zheng, Xinshi Zang, and Martin D.F. Wong. 2021. TopoPart: a Multi-level Topology-Driven Partitioning Framework for Multi-FPGA Systems. In *IEEE/ACM ICCAD*. 1–8. <https://doi.org/10.1109/ICCAD51958.2021.9643481>